Software Design Document

Java Operating System v2.0

Vision

The goal of this project is to develop a modern operating system whose primary programming language is Java. By building the operating system around the Java ecosystem, developers can use familiar tools and the standard Java SDK while benefiting from improved portability, maintainability, and productivity.

The operating system is designed to be modular, secure, and suitable for both desktop and server environments.

Why not just run a JVM on Linux?

Linux has a monolitic kernel. And jxcore is build for a microkernel. It is a new OS for the server. Microkernel delivers a drastically smaller Trusted Computing Base (TCB) and superior fault isolation compared to a monolithic kernel like Linux.

Architecture design:

The operating system is organized into three primary layers.

1. Kernel

The kernel is a lightweight microkernel responsible for the core operating system services.

Responsibilities

- Memory management
- Process and thread scheduling
- Inter-process communication (IPC)
- Device driver interface
- Virtual machine management
- Hardware abstraction

Implementation

- Written in C and Assembly for maximum performance and hardware control
- Supports one or more Java Virtual Machines
- Designed to minimize trusted kernel code

---

2. File System

The file system provides reliable and persistent storage.

Design Goals

- High performance
- Crash resilience
- Audit logging
- Fault tolerance
- Data integrity
- Fast recovery after unexpected shutdowns

---

3. User Interface

The graphical environment is independent of the kernel and runs as user-space services.

Possible desktop environments include:

- OS.js
- Jupyter Notebook
- MyOpenLab

Because the GUI is modular, systems may also run without a graphical interface.

User Interface design:

The operating system supports multiple methods of interaction.

- Command-line interface (CLI)
- Native graphical desktop
- Web-based interface
- SSH remote access

The design emphasizes speed, ease of use, and remote administration.

Compatibility:

To maximize software compatibility, the operating system can host virtual machines that run other operating systems.

Examples include:

- Windows
- Linux

Applications that require another operating system can execute inside isolated virtual machines while the Java operating system remains the host.

Software Development Model

Application developers primarily write software in Java using the standard Java SDK.

Advantages include:

- Large existing ecosystem
- Cross-platform APIs
- Improved developer productivity
- Easier maintenance than low-level system programming
- Strong object-oriented design

Native Code Generation

The execution pipeline is:

1. Write software in Java.
2. Compile Java source code into Java bytecode.
3. Translate or compile bytecode into optimized native machine code for the target processor architecture.
4. Execute the generated native code with minimal runtime overhead.

Supported processor architectures may include:

- x86
- x86-64
- ARM
- RISC-V

Design Goals

The operating system is designed to be:

- Fast
- Secure
- Reliable
- Modular
- Easy to develop for
- Portable across hardware architectures
- Suitable for desktop, embedded, and server systems

Final Product

The final product is a complete Java-based operating system that combines the performance of a microkernel with the productivity of modern Java development.

Users can choose between command-line and graphical environments, manage the system locally or remotely, and run applications or even complete guest operating systems inside virtual machines. The platform aims to provide a secure, extensible, and developer-friendly foundation for next-generation computing.

2026-08-07

Xuyi Wang